OIC: An Online Incentivized and Context-aware Scheme for Task Assignment

Jiamei Ji, Kechao Cai, Zhuoyue Chen, Jinbei Zhang
School of Electronics and Communication Engineering, Sun Yat-sen University, Shenzhen, China
{jijm3,chenzhy225}@mail2.sysu.edu.cn, {caikch3,zhjinbei}@mail.sysu.edu.cn

***Abstract*—Online task assignment is key in areas like crowdsourcing and digital advertising, where context awareness and worker incentives are crucial for quality and sustained engagement. Existing work often focuses on either context-task alignment or basic incentive design. To address both, we propose an Online Incentivized and Context-aware (OIC) task assignment scheme. We model the problem as a Stackelberg game and derive the optimal incentive allocation under an oracle setting. To handle unknown, dynamic task-worker relationships, we develop an online learning algorithm that approximates the oracle solution and provides an upper confidence bound on server utility. Our method achieves sublinear regret and performs close to the oracle in simulations, enabling effective real-time task allocation amid changing conditions and strategic worker behavior.**

***Index Terms*—Task Assignment, Game theory, contextual bandits, incentive mechanism.**

I. Introduction

Task assignment is a fundamental problem in various real-world applications, such as crowdsourcing platforms (e.g., Amazon Mechanical Turk) and online advertising placement (e.g., Google Ads). Efficient task assignment not only enhances productivity but also ensures fair compensation and sustained engagement among workers. In crowdsourcing, properly assigning tasks to workers with suitable expertise can significantly reduce completion time while improving output quality. Similarly, in digital advertising, dynamic ad placement based on task contexts and advertiser characteristics can lead to higher click-through rates.

To expand the applicability of the task allocation scheme, task assignment in context-aware settings and incentivized settings presents two critical challenges. In context-aware settings, achieving optimal task assignment requires aligning task requirements with worker capabilities, compounded by time-varying contexts (e.g., shifting task demands). In incentivized settings, task assignment schemes must account for self-interested workers and design incentive mechanisms to ensure workers' non-negative utility. Additionally, many applications aim to promote broader goals, such as social welfare and truthful reporting.

To tackle context-awareness, contextual bandit algorithms have emerged as powerful tools. A rich body of work exists in this area [1]–[5], and many recent task assignment studies have adapted contextual bandits to dynamic environments [6]–[9]. Although these approaches perform well in online task assignment scenarios, they largely overlook the strategic behavior of workers and therefore lack integrated incentive mechanisms. Conversely, a parallel line of work focuses on incentivized schemes, leveraging tools from game theory and auction theory [10]–[13]. These approaches explicitly model strategic agents but typically assume static or fully known environments and lack the adaptability needed to handle dynamic and uncertain contexts.

To bridge this gap, we integrate contextual bandits into a game-theoretic framework with multiple strategic workers. We formulate the one-to-many task assignment problem as a Stackelberg game. The context-dependent and unknown service quality makes computing the exact equilibrium for the Stackelberg game intractable. To overcome this, we employ online learning to approximate the equilibrium, introducing an exploration-exploitation tradeoff. We manage this tradeoff by establishing an upper confidence bound (UCB) on the server's utility and show that our method achieves strong long-term learning performance. Our main contributions are summarized as follows:

- We propose a novel online task assignment framework that accounts for dynamic task context and the heterogeneity of workers.
- We formulate the incentive mechanism design as a Stackelberg game. In the oracle setting, which assumes prior knowledge of service quality, we derive the optimal solution to the Stackelberg game.
- Building on the oracle strategy, we develop an incentivized online task assignment scheme and prove that it achieves a sublinear regret bound.
- We evaluate the proposed algorithm through extensive simulations and show that it effectively approximates the oracle benchmark while maximizing the server's long-term utility.

The remainder of this paper is organized as follows. We discuss the related works in Section II. Section III introduces the system model and formulates the problem. Section IV presents our proposed scheme and its theoretical analysis. We evaluate the performance through numerical experiments in Section V, and finally conclude the paper in Section VI.

II. Related Work

A. Context-aware Task Assignment

Context-aware task assignment leverages rich contextual information to better match tasks with workers, enhancing

completion performance [9], [14]–[18]. In this settings, contextual bandit formulations have proven effective for addressing problems involving contextual tasks and heterogeneous workers [6]–[8]. For example, An *et al.* apply a contextual bandit approach for quality-aware crowdsensing by modeling task and worker attributes [6]. Compared to [6], the method in [8] have been shown to achieve a sublinear regret bound, thereby ensuring strong online learning performance. Specifically, Elahi *et al.* propose an adaptive optimistic matching algorithm that discretizes the joint task-worker context space and achieves a sublinear regret bound of $\tilde{O}(T^{(\bar{D}+1)/(\bar{D}+2)})$, where $\bar{D}$ denotes a notion of dimensionality that captures the structural complexity of the problem [8].

Compared with these prior works, we incorporate contextual bandits within a more complex multi-worker game-theoretic framework that includes simultaneous incentive mechanism design, achieving a regret bound of $O(\sqrt{T}\log T)$.

B. Incentivized Task Assignment

Incentivized task assignment schemes typically fall into two primary modes: Worker Selected Tasks (WST) and Server Assigned Tasks (SAT), each targeting different objectives and employing distinct mechanisms [19], [20]. In WST, tasks are published openly, and workers choose based on personal utility or preference [21], [22]. In contrast, SAT features centralized assignment by the server to optimize system-level objectives such as social welfare [23], fairness [24], and truthfulness [25], [26]. SAT approaches commonly employ auction-based mechanisms [11], [13] or game-theoretic models like Stackelberg games [10], [12].

Our task assignment model adopts the SAT mode and is most closely related to the frameworks presented in [10], [27]. Yang *et al.* propose a server-centric model for crowdsensing, optimizing payments based on selfish worker behavior, while Wang *et al.* extend this to federated learning by factoring in data quality. However, both of them assume static task contexts, treating worker performance as fixed. In contrast, we consider task performance to depend on unknown and dynamic interactions between task context and worker properties. This complexity motivates our use of online learning to approximate the Stackelberg game solution and design an adaptive incentive scheme.

III. System Model and Problem Formulation

In this section, we introduce the proposed online task assignment framework, define the utilities of both the server and clients, and formally present the objective of our online scheme.

A. Online Task Assignment Framework

Our online task assignment framework, illustrated in Fig. 1, models a dynamic interaction between a task publisher (server) and multiple clients (workers). The server assigns a task to selected clients and offers corresponding incentives. Clients then execute the task, which involves interacting with the external environment.

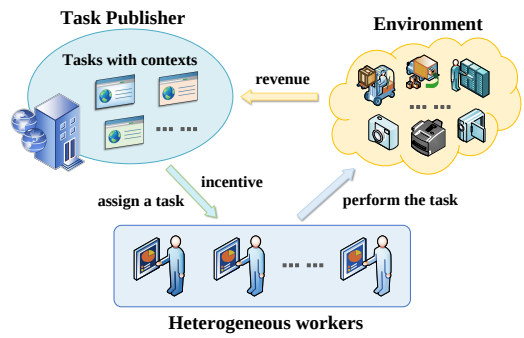


Fig. 1. Online incentivized and context-aware task assignment framework.

Formally, the system consists of a server and a set of clients $\mathcal{N}$, where $|\mathcal{N}| = N$. In each round, the server selects one task from a set of K candidate tasks and assigns it to clients to maximize its own utility. Each client $i \in \mathcal{N}$ incurs a unit cost c_i to perform a task, and provides a service quality k is denoted as $q_{i,t}^k$ for task k in round t, which could represent metrics such as click-through rate (CTR) or revenue. This framework operates as a multi-round online learning process over a total of T rounds. In each round $t \in [T]$, the following steps occur:

(1) The server observes a global action set, i.e., the context vectors of the K candidate tasks: $\mathcal{A}_t = \{\boldsymbol{x}_t^1, \cdots, \boldsymbol{x}_t^k, \cdots, \boldsymbol{x}_t^K\} \in \mathbb{R}^d$. The d-dimensional vector $\boldsymbol{x}_t^k$ represents the context vector of task $k \in [K]$ at round t.

(2) The server selects an optimal task k_t to be performed at round t, determines the total payment P_t, and sallocates a required resource level $r_{i,t} \geq 0$ to each client $i \in \mathcal{N}$ based on the clients' costs and service qualities. A client participates in the task if $r_{i,t} > 0$.

(3) Let $\mathcal{S}_t$ denote the set of participating clients. Each client $i \in \mathcal{S}_t$ executes the task $\boldsymbol{x}_t$ according to its allocated resources $r_{i,t}$.

However, in real-world applications, the server does not have prior knowledge of each client's service quality. Therefore, we assume that service quality of each clientis an unknown but fixed linear function of the task context, with parameters that can be learned over time. Specifically, for each client $i \in \mathcal{N}$, the actual service quality $q_{i,t}^k$ for task k in round t is given by:

$$q_{i,t}^k = \langle \boldsymbol{x}_t^k, \boldsymbol{\theta}_i \rangle + \eta_{i,t}^k, \tag{1}$$

where $\boldsymbol{\theta}_i \in \mathbb{R}^d$ is the unknown model parameter for client i and $\eta_{i,t}^k$ is a zero-mean σ-subgaussian noise term. Over time, each client can estimate its parameter $\boldsymbol{\theta}_i$ using historical data and thus estimate the service quality $\hat{q}_{i,t}^k$. This estimated value enables the server to approximate the optimal solution to the task assignment problem in the absence of complete information.

B. Utility Definition

To design an effective incentive mechanism, we define the utility functions of both the clients and the server based on true service quality.

1) Client's Utility: The utility of a client is defined as the payment received from the server minus the incurred cost. When the server assigns task k to client i at round t, the client's utility is:

$$u_{i,t}^k = \frac{q_{i,t}^k r_{i,t}^k}{\sum_{j\in\mathcal{S}_t} q_{j,t}^k r_{j,t}^k} P_t^k - r_{i,t}^k c_i \tag{2}$$

where P_t^k is the total payment for task k at round t, and $r_{i,t}^k$ is resource level requested from client i for task k at round t.

2) Server's Utility: If the server requires clients to perform task k, its utility is defined as

$$u_{0,t}^k = \gamma \log(1 + \sum_{i\in\mathcal{N}} q_{i,t}^k r_{i,t}^k) - P_t^k \tag{3}$$

where $\gamma > 1$ is a system parameter. The logarithmic function captures diminishing returns with respect to the total contributed service quality.

C. Objective

Our online task assignment framework is server-centric, aims to maximize server's utility while ensuring clients are incentivized to participate. Specifically, at round t, for each task $k \in [K]$, we compute the Nash equilibrium of the clients' strategies and then determine the server's utility under that equilibrium. Nash equilibrium is defined as follows.

Definition 1 (Nash Equilibrium, NE)**.** *A Nash equilibrium is a strategy profile* $\boldsymbol{r}_t^{k^{\mathrm{NE}}} = (r_{1,t}^{k^{\mathrm{NE}}}, r_{2,t}^{k^{\mathrm{NE}}}, ..., r_{N,t}^{k^{\mathrm{NE}}})$ *in which no client can increase its utility by unilaterally deviating from its strategy. Formally, for each client* $i \in \mathcal{N}$ *and any* $r_{i,t}^k \geq 0$*,*

$$u_{i,t}^k(r_{i,t}^{k^{\mathrm{NE}}}, r_{-i,t}^{k^{\mathrm{NE}}}) \geq u_{i,t}^k(r_{i,t}^k, r_{-i,t}^{k^{\mathrm{NE}}}), \tag{4}$$

where $r_{-i,t}^{k^{\mathrm{NE}}}$ *denotes the strategy profile of all clients except* i*.*

Given the optimal solution for each task, the server then selects the task that maximizes its utility. The overall objective is to minimize the cumulative regret:

$$R_T = \sum_{t=1}^{T} u_{0,t}^{k_t^*} - u_{0,t}^{k_t}, \tag{5}$$

where k_t^* is the optimal task at round t, and k_t is the task chosen by the server. This regret captures the performance gap between the optimal and actual task selections across all rounds. The lower the regret, the better our online scheme performs.

IV. OIC: Online Incentivized and Context-aware Task Assignment Scheme

The proposed framework is formulated as a Stackelberg game, where the server acts as the leader and the clients act as followers. To derive an optimal solution to this task assignment problem, we decompose the scheme into the following three sub-problems:

- **P1 Unique Nash Equilibrium with Known Service Quality:** Assume that the system has perfect information about the clients' service qualities, i.e., $q_{i,t}^k$ is known for all clients and tasks. For a given task k and total payment P_t^k, the server needs to determine a unique resource allocation strategy profile $\boldsymbol{r}_t^{k^{\mathrm{NE}}}$ that satisfies the Nash Equilibrium for all clients. (Discussed in Section IV-A)
- **P2 Approximate NE under Uncertainty via Online Learning:** The server needs to obtain estimates of service quality via online learning. Using these estimates, the server computes an approximate Nash Equilibrium strategy profile $\hat{\boldsymbol{r}}_t^{k^{\mathrm{NE}}}$ for each task k, which approximates the equilibrium obtained in **P1**. (Discussed in Section IV-B)
- **P3 Optimal Task and Payment Selection:** Given the estimated NE strategy profile, the server estimates the corresponding optimal payment $\hat{P}_t^k$ for each task $k \in [K]$ that maximizes its utility. The server then evaluates its estimated utility $\hat{u}_{0,t}^k$ for each task and selects the task with the highest expected utility, i.e., $k_t = \arg\max_k \hat{u}_{0,t}^k$. (Discussed in Section IV-B)

In the following paragraphs, we present solutions to these sub-problems in detail and provide a performance analysis of the proposed online incentive mechanism in Section IV-C.

A. Oracle: NE with Perfect Information of Service Quality

To obtain the solution under an unknown service quality setting, we first derive the Nash Equilibrium (NE) assuming the system has perfect prior knowledge of service quality, referred to as the oracle.

To analyze the optimal strategy of client i for task k that satisfies the Nash equilibrium, we compute the derivative of $u_{i,t}^k$ with respect to $r_{i,t}^k$:

$$\frac{\partial u_{i,t}^k}{\partial r_{i,t}^k} = \frac{q_{i,t}^k \sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{\left(\sum_{j\in\mathcal{N}} q_{j,t}^k r_{j,t}^k\right)^2} P_t^k - c_i.$$

The second derivative of $u_{i,t}^k$ with respect to $r_{i,t}^k$ is:

$$\frac{\partial^2 u_{i,t}^k}{\partial (r_{i,t}^k)^2} = -\frac{2{q_{i,t}^k}^2 \Sigma_{j\in N\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{(\Sigma_{j\in N} q_{j,t}^k r_{j,t}^k)^3} P_t^k < 0,$$

which imply that $u_{i,t}^k$ is concave with respect to $r_{i,t}^k$. Therefore, the optimal strategy of client i for task k corresponds to the solution of $\frac{\partial u_{i,t}^k}{\partial r_{i,t}^k} = 0$:

$$r_{i,t}^k = \frac{\sqrt{\frac{q_{i,t}^k P_t^k \sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{c_i}} - \sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{q_{i,t}^k}$$

If the right-hand side of the above equation is negative, we set $r_{i,t}^k = 0$ to prevent negative resource allocation. Therefore the optimal strategy of client i for task k is:

$$\beta_{i,t}^k(r_{-i,t}^k) = \begin{cases} 0 & P_t^k \leq \frac{c_i}{q_{i,t}^k}\Sigma, \\ \frac{\sqrt{\frac{q_{i,t}^k P_t^k \Sigma}{c_i}} - \Sigma}{q_{i,t}^k} & otherwise, \end{cases} \tag{6}$$

where $\Sigma = \sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k$

However, the solution given in (6) is not explicit, as it contains terms $r_{j,t}^k$ for all clients $j \in \mathcal{N} \setminus \{i\}$ that remain unsolved. To facilitate the design of an algorithm for computing an NE, we establish that the NE strategy profile satisfies four statements in Theorem 1. The proof of Theorem 1 is provided in Appendix A in our full version of the paper [28].

Theorem 1 (Properties of NE)**.** *Assume that P_t^k is given. Let $\tilde{\boldsymbol{r}}_t^k = (\tilde{r}_{1,t}^k, \tilde{r}_{2,t}^k, ..., \tilde{r}_{N,t}^k)$ be the strategy profile of an NE for the Stackelberg game of task k, and let $\tilde{\mathcal{S}}_t^k$ denote the set of selected clients, i.e., $\tilde{\mathcal{S}}_t^k = \{i \in \mathcal{N} | \tilde{r}_{i,t}^k > 0\}$. The following four statements hold:*

(1) $|\tilde{\mathcal{S}}_t^k| \geq 2$.

(2) $\tilde{r}_{i,t}^k = \begin{cases} 0 & i \notin \tilde{\mathcal{S}}_t^k \\ \frac{(|\tilde{\mathcal{S}}_t^k|-1)P_t^k}{q_{i,t}^k \sum_{j\in s_t^k} \frac{c_j}{q_{j,t}^k}}\left(1 - \frac{(|\tilde{\mathcal{S}}_t^k|-1)c_i}{q_{i,t}^k \sum_{j\in s_t^k} \frac{c_j}{q_{j,t}^k}}\right) & otherwise. \end{cases}$

(3) If $\frac{c_i}{q_{i,t}^k} \leq \max_{j\in\tilde{\mathcal{S}}_t^k}\{\frac{c_j}{q_{j,t}^k}\}$*, then* $i \in \tilde{\mathcal{S}}_t^k$.

(4) Assume that the users are ordered such that $\frac{c_{i(1)}}{q_{i(1),t}^k} < \frac{c_{i(2)}}{q_{i(2),t}^k} < \cdots < \frac{c_{i(N)}}{q_{i(N),t}^k}$*. Let h be the largest integer in $[2, N]$ such that* $\frac{c_{i(h)}}{q_{i(h),t}^k} < \frac{\sum_{j=1}^h \frac{c_{i(j)}}{q_{i(j),t}^k}}{h-1}$*. Then* $\tilde{\mathcal{S}}_t^k = \{i^{(1)}, i^{(2)}, ..., i^{(h)}\}$.

Theorem 1 provides client selection criteria and an explicit form of the NE solution. The first statement indicates that at least two clients are required to form an equilibrium. The second statement presents an explicit expression for the equilibrium. The third and fourth statements imply a preference for high-quality, low-cost clients in the equilibrium and confirm the uniqueness of the Stackelberg equilibrium.

Leveraging Theorem 1, we propose Algorithm 1 to compute the NE for task k The algorithm first sorts clients by their cost-to-quality ratio and then selects clients with the lowest ratios until the condition in Line 4 is satisfied. We prove that the strategy profile $\boldsymbol{r}_t^{k^{\mathrm{NE}}}$ computed by Algorithm 1 is the NE of the Stackelberg game of task k in Theorem 2. The proof of Theorem 2 is provided in Appendix B [28].

Theorem 2. *The strategy profile $\boldsymbol{r}_t^{k^{\mathrm{NE}}}$ computed by Algorithm 1 is the NE of the Stackelberg game of task k, which corresponds to the solution in* (6)*.*

B. Online Scheme with Unknown Service Quality

Based on the oracle, we propose the **Online Incentivized and Context-aware scheme** (OIC) for task assignment, which is shown in Algorithm 2 and 3.

1) Optimistic Estimate of Context-aware Service Quality: To estimate service quality, we first need to estimate the model parameters $\boldsymbol{\theta}_i$ so that the expected service quality can be computed according to the linear model defined in (1).

To estimate the model $\boldsymbol{\theta}_i$, we need to collect the action history and reward history of each client i. Therefore, client i

Algorithm 1 Oracle: NE computation of task k

1: **Input:** P_t^k, c_i, $q_{i,t}^k$, $i \in \mathcal{N}$.
2: Sort clients according to their cost/quality ratio, $\frac{c_{i(1)}}{q_{i(1),t}^k} < \frac{c_{i(2)}}{q_{i(2),t}^k} < \cdots < \frac{c_{i(N)}}{q_{i(N),t}^k}$
3: $\mathcal{S}_t^k \leftarrow \{i^{(1)}, i^{(2)}\}, n_{cur} \leftarrow 3, i \leftarrow i^{(n_{cur})}$
4: **while** $n_{cur} \leq N$ and $\frac{c_{i(n_{cur})}}{q_{i(n_{cur}),t}^k} < \frac{\frac{c_{i(n_{cur})}}{q_{i(n_{cur}),t}^k} + \sum_{j\in\mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}{|\mathcal{S}_t^k|}$ **do**
5: $\quad \mathcal{S}_t^k \leftarrow \mathcal{S}_t^k \cup \{i^{(n_{cur})}\}, n_{cur} \leftarrow n_{cur} + 1, i \leftarrow i^{(n_{cur})}$
6: **end while**
7: **for all** $i \in \mathcal{N}$ **do**
8: $\quad$ **if** $i \in \mathcal{S}_t$ **then**
9: $\quad\quad r_{i,t}^{k^{\mathrm{NE}}} = \frac{(|\mathcal{S}_t^k|-1)P_t^k}{q_{i,t}^k \sum_{j\in\mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}\left(1 - \frac{(|\mathcal{S}_t^k|-1)c_i}{q_{i,t}^k \sum_{j\in\mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}\right)$
10: $\quad$ **else**
11: $\quad\quad r_{i,t}^{k^{\mathrm{NE}}} = 0$
12: $\quad$ **end if**
13: **end for**
14: **return** $\boldsymbol{r}_t^{k^{\mathrm{NE}}} = (r_{1,t}^{k^{\mathrm{NE}}}, r_{2,t}^{k^{\mathrm{NE}}}, ..., r_{N,t}^{k^{\mathrm{NE}}})$

maintains a covariance matrix $\boldsymbol{V}_{i,t}$ and a reward vector $\boldsymbol{v}_{i,t}$, which store the client's action and reward history, respectively:

$$\boldsymbol{V}_{i,t} = \lambda \boldsymbol{I} + \sum_{\tau\leq t, r_{i,\tau}>0} \boldsymbol{x}_\tau \boldsymbol{x}_\tau^\top,$$

$$\boldsymbol{v}_{i,t} = \sum_{\tau\leq t, r_{i,\tau}>0} \boldsymbol{x}_\tau q_{i,\tau}.$$

At the beginning of the first round, we initialize $\boldsymbol{V}_{i,t} = \lambda \boldsymbol{I}$ and $\boldsymbol{v}_{i,t} = \boldsymbol{0}^d$, where λ the regularization parameter, and $\boldsymbol{I}$ is the identity matrix (see Line 1 in Algorithm 2). Then use the following two equations to iteratively update the covariance matrix and reward vector at each round t (Line 13 of Algorithm 2):

$$\boldsymbol{V}_{i,t} = \begin{cases} \boldsymbol{V}_{i,t-1} + \boldsymbol{x}_t \boldsymbol{x}_t^\top, & \hat{r}_{i,t} > 0 \\ \boldsymbol{V}_{i,t-1}, & \hat{r}_{i,t} = 0. \end{cases} \tag{7}$$

$$\boldsymbol{v}_{i,t} = \begin{cases} \boldsymbol{v}_{i,t-1} + \boldsymbol{x}_t q_{i,t}, & \hat{r}_{i,t} > 0 \\ \boldsymbol{v}_{i,t-1}, & \hat{r}_{i,t} = 0. \end{cases} \tag{8}$$

Using these historical data, we estimate $\boldsymbol{\theta}_i$ via ridge regression:

$$\hat{\boldsymbol{\theta}}_{i,t} = \boldsymbol{V}_{i,t-1}^{-1} \boldsymbol{v}_{i,t-1}. \tag{9}$$

Then with the estimated model $\hat{\boldsymbol{\theta}}_{i,t}$, an estimated service quality $\langle \boldsymbol{x}_t^k, \hat{\boldsymbol{\theta}}_{i,t} \rangle$ of client i can be computed for task k at round t.

However, directly using the estimated service quality for client selection may exclude clients with high actual quality but limited exploration. In such cases, the variance of the estimate may be large, resulting in an underestimated service quality. To balance exploration and exploitation, our scheme uses an optimistic estimate of service quality by computing

Algorithm 2 OIC

1: **[Clients]** Each client $i \in \mathcal{N}$ initializes $\boldsymbol{V}_{i,0} = \lambda \boldsymbol{I}$, $\boldsymbol{v}_{i,0} = \boldsymbol{0}^d$ and sends c_i to the server.
2: **for all** $t \in \mathcal{T}$ **do**
/ Optimistic Estimate of Context-aware Service Quality */*
3: **[Server]** Receive $\mathcal{A}_t$ from the environment and send $\boldsymbol{x}_t$ to clients.
4: **[Clients]** Receive $\mathcal{A}_t$ from the server, compute optimistic estimate $\hat{q}_{i,t}^k$ according to (10), and send $\hat{q}_{i,t}^k$ to the server.
5: **[Server]** Receive $\hat{q}_{i,t}^k$ from clients $i \in \mathcal{N}$.
/ Approximate optimal solution of each task*/*
6: **for all** $k \in [K]$ **do**
7: **[Server]** Compute $\hat{u}_{0,t}^k$, $\hat{P}_t^k$ and $\hat{\boldsymbol{r}}_t^{k^{\mathrm{NE}}}$ using Algorithm 3.
8: **end for**
/ Optimal Task Decision*/*
9: **[Server]** Select the task $k_t = \arg\max_k \hat{u}_{0,t}^k$
/ Incentivized Task Execution*/*
10: **[Server]** Assign task $\boldsymbol{x}_t = \boldsymbol{x}_t^{k_t}$ to clients with resource requirement $\hat{\boldsymbol{r}}_t = \hat{\boldsymbol{r}}_t^{k_t^{\mathrm{NE}}}$, and distributed payment $\hat{P}_t = \hat{P}_t^{k_t}$ to clients.
11: **[Clients]** Participate in the assigned task according to $\hat{\boldsymbol{r}}_t$ and observe the true service quality $q_{i,t}$.
12: **[Server]** Get revenue after clients performing the task.
13: **[Clients]** Update $\boldsymbol{V}_{i,t}$ and $\boldsymbol{v}_{i,t}$ for each client $i \in \mathcal{N}$ according to (7) and (8).
14: **end for**

an Upper Confidence Bound (UCB) [2], [29] (see Line 4 in Algorithm 2):

$$\hat{q}_{i,t}^k = \langle \boldsymbol{x}_t^k, \hat{\boldsymbol{\theta}}_{i,t} \rangle + \alpha_{i,t} \|\boldsymbol{x}_t^k\|_{\boldsymbol{V}_{i,t}^{-1}}, \tag{10}$$

where $\alpha_{i,t} = \sigma\sqrt{\log \frac{|\boldsymbol{V}_{i,t}|}{|\lambda \boldsymbol{I}|} + 2\log\frac{2}{\delta}} + \sqrt{\lambda}$ and δ denotes the confidence level. In this formulation, the first term represents the expected service quality, while the second term provides a confidence interval with at least $1-\delta$ probability. By using this optimistic estimate, our scheme naturally prioritizes clients with high expected quality (exploitation) or high uncertainty (exploration).

2) Approximate Optimal Solution of Each Task: After obtaining the optimistic estimates of service quality, our goal is to compute an approximate optimal solution each task k, aiming to maximize the server's utility while approximating the NE. Specifically, we aim to determine the undetermined variables $\hat{\boldsymbol{r}}_t^{k^{\mathrm{NE}}}$ and $\hat{P}_t^k$.

To extend the NE computed using Algorithm 1 in the oracle setting to the online setting, we incorporate the optimistic estimate of service quality derived in (10). By replacing $q_{i,t}^k$ with the optimistic counterpart $\hat{q}_{i,t}^k$ in Algorithm 1, we obtain Algorithm 3, which computes an approximate NE.

Once the resource allocation $\hat{\boldsymbol{r}}_t^{k^{\mathrm{NE}}}$ is determined (see Line 10 in Algorithm 3), we proceed to compute the optimal

Algorithm 3 Approximate optimal solution of task k

1: **Input:** $\{c_i, \hat{q}_{i,t}^k\}_{i\in\mathcal{N}}$.
2: Sort clients according to their cost/quality ratio, $\frac{c_{i(1)}}{\hat{q}_{i(1),t}^k} < \frac{c_{i(2)}}{\hat{q}_{i(2),t}^k} < \cdots < \frac{c_{i(N)}}{\hat{q}_{i(N),t}^k}$
3: $\hat{\mathcal{S}}_t^k \leftarrow \{i^{(1)}, i^{(2)}\}, n_{cur} \leftarrow 3, i \leftarrow i^{(n_{cur})}$
4: **while** $n_{cur} \le N$ and $\frac{c_{i(n_{cur})}}{\hat{q}_{i(n_{cur}),t}^k} < \frac{\frac{c_{i(n_{cur})}}{\hat{q}_{i(n_{cur}),t}^k} + \sum_{j\in\hat{\mathcal{S}}_t^k}\frac{c_j}{\hat{q}_{j,t}^k}}{|\hat{\mathcal{S}}_t^k|}$ **do**
5: $\hat{\mathcal{S}}_t^k \leftarrow \hat{\mathcal{S}}_t^k \cup \{i^{(n_{cur})}\}, n_{cur} \leftarrow n_{cur}+1, i \leftarrow i^{(n_{cur})}$
6: **end while**
7: Compute $\hat{X}_t^k = \frac{(|\hat{\mathcal{S}}_t^k|-1)}{\sum_{j\in\hat{\mathcal{S}}_t^k}\frac{c_i}{\hat{q}_{i,t}^k}}$, $\hat{P}_t^k = \gamma - \frac{1}{\hat{X}_t^k}$ and $\hat{u}_{0,t}^k = \gamma\log(\gamma\hat{X}_t^k) + \frac{1}{\hat{X}_t^k} - \gamma$
8: **for all** $i \in \mathcal{N}$ **do**
9: **if** $i \in \hat{\mathcal{S}}_t$ **then**
10: $\hat{r}_{i,t}^{k^{\mathrm{NE}}} = \frac{(|\hat{\mathcal{S}}_t^k|-1)\hat{P}_t^k}{\hat{q}_{i,t}^k\sum_{j\in\hat{\mathcal{S}}_t^k}\frac{c_j}{\hat{q}_{j,t}^k}}(1 - \frac{(|\hat{\mathcal{S}}_t^k|-1)c_i}{\hat{q}_{i,t}^k\sum_{j\in\hat{\mathcal{S}}_t^k}\frac{c_j}{\hat{q}_{j,t}^k}})$
11: **else**
12: $\hat{r}_{i,t}^{k^{\mathrm{NE}}} = 0$
13: **end if**
14: **end for**
15: **return** $\hat{u}_{0,t}^k$, $\hat{P}_t^k$ and $\hat{\boldsymbol{r}}_t^{k^{\mathrm{NE}}} = (\hat{r}_{1,t}^{k^{\mathrm{NE}}}, \hat{r}_{2,t}^{k^{\mathrm{NE}}}, ..., \hat{r}_{N,t}^{k^{\mathrm{NE}}})$

payment $\hat{P}_t^k$ that maximize the server utility. Applying the value of $\hat{\boldsymbol{r}}_t^{k^{\mathrm{NE}}}$ computed in Line 10 in Algorithm 3, we can compute the approximate server utility $\hat{u}_{0,t}^k$ as follows:

$$\hat{u}_{0,t}^k = \gamma\log(1 + \frac{(|\hat{\mathcal{S}}_t^k|-1)\hat{P}_t^k}{\sum_{j\in\hat{\mathcal{S}}_t^k}\frac{c_i}{\hat{q}_{i,t}^k}}) - \hat{P}_t^k$$

Let $\hat{X}_t^k = \frac{(|\hat{\mathcal{S}}_t^k|-1)}{\sum_{j\in\hat{\mathcal{S}}_t^k}\frac{c_i}{\hat{q}_{i,t}^k}}$, the server utility can be rewritten as:

$$\hat{u}_{0,t}^k = \gamma\log(1 + \hat{X}_t^k\hat{P}_t^k) - \hat{P}_t^k.$$

We can analyze the server utility $\hat{u}_{0,t}^k$ as a function of $\hat{P}_t^k$. The first derivative of $\hat{u}_{0,t}^k$ with respect to $\hat{P}_t^k$ is:

$$\frac{\partial\hat{u}_{0,t}^k}{\partial\hat{P}_t^k} = \frac{\gamma}{1+\hat{X}_t^k\hat{P}_t^k} - 1. \tag{11}$$

The second derivative of $\hat{u}_{0,t}^k$ with respect to $\hat{P}_t^k$ is:

$$\frac{\partial^2\hat{u}_{0,t}^k}{\partial(\hat{P}_t^k)^2} = -\frac{\gamma\hat{X}_t^k}{(1+\hat{X}_t^k\hat{P}_t^k)^2} < 0.$$

This shows that $\hat{u}_{0,t}^k$ is a concave function of $\hat{P}_t^k$ with respect to the payment $\hat{P}_t^k$. To find the optimal payment $\hat{P}_t^k$, we set (11) to zero and get

$$\hat{P}_t^k = \gamma - \frac{1}{\hat{X}_t^k}. \tag{12}$$

Then the approximate server utility is:

$$\hat{u}_{0,t}^k = \gamma\log(\gamma\hat{X}_t^k) + \frac{1}{\hat{X}_t^k} - \gamma.$$

We apply (12) in Line 7 of Algorithm 3 to complete our solution.

3) Optimal Task decision: After computing the approximate optimal solution for each task k, we select the task k_t that yields the highest server utility as the optimal task to assign to clients.

However, the challenge in task selection lies in the uncertainty of the server utility, due to the unknown nature of service quality. As a result, the task selection process must also balance the tradeoff between exploration and exploitation. To address this, we select the task based on the upper confidence bound of the server utility. Specifically, we analyze the relationship between the approximate server utility $\hat{u}_{0,t}^k$ and the actual utility $u_{0,t}^k$ and prove that $\hat{u}_{0,t}^k$ serves as a valid UCB of $u_{0,t}^k$.

This result is formalized in Theorem 3, which enables us to choose the task with the maximum $\hat{u}_{0,t}^k$ as the optimal task to be assigned to clients (see Line 9 in Algorithm 2).

Theorem 3 (Upper Confidence Bound of Server Utility)**.** *If $\gamma \geq (1+\frac{1}{N-1})\frac{c_{\max}}{q_{\min}}$ holds, then the approximate server utility $\hat{u}_{0,t}^k$ obtained by Algorithm 2 is an upper confidence bound ofthe actual server utility $u_{0,t}^k$ with probability at least $(1-\delta)^N$.*

Proof sketch: First, we prove that $\hat{X}_t^k$ is an upper confidence bound of $X_t^k = \frac{(|\mathcal{S}_t^k|-1)}{\sum_{j\in\mathcal{S}_t^k}\frac{c_i}{q_{i,t}^k}}$, where $\mathcal{S}_t^k$ is the set of selected clients obtained using the autual service quality. Then, under the condition of sufficiently large γ, we reduce the problem of bounding $\hat{u}_{0,t}^k$ to that of bounding $\hat{X}_t^k$ being an upper confidence bound of X_t^k, thus completing the proof. The full proof is provided in Appendix C [28].

4) Incentivized Task Execution: Once the optimal task k_t is selected, the server assigns the task $\boldsymbol{x}_t^{k_t}$ to the clients with resource requirement $\hat{\boldsymbol{r}}_t^{k_t^{\mathrm{NE}}}$ and distributed payment $\hat{P}_t^{k_t}$ to clients. Each client participates in the task according to the assigned resource requirement, and observes their actual service quality $q_{i,t}$. Upon completion of the task, the server receives revenue based on the clients' performance. The pseudocode for the incentivized task execution process is shown in Lines 10-12 of Algorithm 2.

C. Performance Analysis

In this section, we analyze the performance of the proposed scheme. Specifically, we establish that the regret of our online scheme grows sublinearly with the number of rounds T, indicating that the scheme becomes increasingly effective over time. This result is formalized in Theorem 4.

Theorem 4 (Regret Bound)**.** *Suppose that $||\boldsymbol{x}_t||_2 \leq L$ and $\gamma \geq (1+\frac{1}{N-1})\frac{c_{\max}}{q_{\min}}$. Then, the regret of Algorithm 2 is bounded as follows with probability at least $(1-\delta)^N$. The regret is bounded by:*

$$R_T \leq O(\frac{\gamma N \sigma d}{q_{\min}}\sqrt{T}\log(TL^2/\lambda d)), \tag{13}$$

where $q_{\min}$ is the minimum value of data quality, d is the dimension of the feature vector of tasks, and δ is the confidence parameter.

Proof Sketch: In this section, we provide the proof of Theorem 4. First, we bound the single-round regret. Specifically, we show that the regret in the server's utility in a given round can be bounded by the uncertainty, i.e., the confidence bounds, of the clients' utility estimates in that round. This result establishes a crucial connection between the server's performance and the quality of client-side task assignments. Second, we bound the cumulative regret of the server's utility. This is done by leveraging the confidence bounds on service quality to construct a martingale sequence, which enables us to show that the sum of the confidence bounds over time converges. By combining these two steps, we derive the overall sublinear regret bound. The complete proof is provided in Appendix D [28].

V. Experiments

We conduct comprehensive experiments on both synthetic and real-world datasets to evaluate the performance of our proposed online strategy. The experiments aim to:

- Validate the correctness of the Nash Equilibrium (NE) computation in the oracle;
- Compare our online task assignment strategy with its offline counterpart and analyze the performance gap;
- Evaluate long-term performance and compare it with two baseline methods.

A. Experiment Settings

We conduct experiments on two datasets: a synthetic dataset and a real-world dataset (Yahoo "Learning to Rank Challenge" dataset [30]). The detailed parameter settings are as shown in TABLE. I.

Synthetic dataset: The synthetic dataset is generated based on the linear model of service quality. Each task context $\boldsymbol{x}_t^k$ is sampled independently from a uniform distribution over $[0,1]^d$. Each client's model parameter $\boldsymbol{\theta}_i$ is also sampled uniformly from $[0,1]^d$. The true service quality is calculated as the inner product between the task context and the corresponding client's model.

Yahoo dataset: The Yahoo dataset is a real-world dataset that contains information about users' interests and their interactions with ads. We use "set2.test.txt" of Yahoo dataset in our experiments, which includes 103,174 rows and 702 columns. The first column provides user ratings (ranging from 0 to 4), which we treat as the service quality. The second column is the query ID, and the remaining 700 columns are normalized feature values in the [0,1] range. To reduce the computational burden, we select 9 feature columns with highest number of valid entries. We cluster features into 500 groups using K-means clustering to simulate the model similarity among different clients, and then randomly select N groups of features to construct N model parameters for N clients. For client $i \in [N]$, we compute its model parameter $\boldsymbol{\theta}_i$ by taking the ratio of the total relevance score to the total feature sum across the group. The remaining feature groups are used to form task contexts over T rounds. Each client's actual service quality in a given round is computed as the inner

TABLE I
PARAMETER SETTINGS OF SYNTHETIC DATASET AND YAHOO DATASET

Parameters	Value
Total number of rounds: T	10^3
Number of tasks/ads: K	10
Number of total clients: N	20
Dimension of x_t^k or θ_i: d	9
Bound of $\|x_t^k\|_2$: L	9
Confidence probability: $1-\delta$	0.95
Minimum cost value: $\underline{C}$	0.1
Maximum cost value: $\overline{C}$	1.1
Ratio of virtual utility and real-world payment: γ	5

product between their model parameter and the current task context.

B. Verification of NE Computation in Oracle

We verify the correctness of the Nash Equilibrium (NE) computation in Algorithm 1. To do so, we plot each client's utility as a function of its allocated resource while fixing the resources of all other clients to the equilibrium values computed by Algorithm 1. The equilibrium points identified by the algorithm are marked with triangles in Fig. 2. As shown in Fig. 2, the utility curve for each client exhibits a concave shape. The utility reaches its maximum precisely at the point corresponding to the resource allocation from the Algorithm 1. When a client deviates, either increasing or decreasing its resource usage, its utility decreases. This behavior confirms that no client has an incentive to unilaterally deviate from its equilibrium strategy, satisfying the definition of NE. Therefore, OIC effectively rationalizes clients' participation in the assigned task in alignment with the server's expectations.

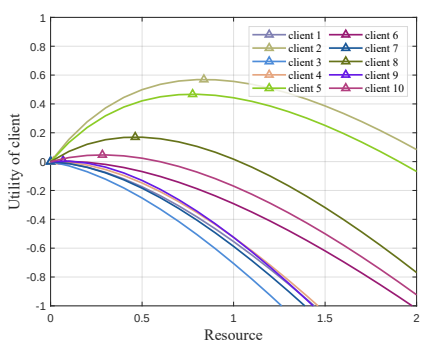


Fig. 2. Verification of NE computation.

C. Gap between Online strategy and Oracle

We evaluate the performance of the proposed online strategy by comparing the approximate NE computed via Algorithm 3 (online scheme) with the exact NE computed via Algorithm 1 (oracle). We conduct this comparison across the following three metrics: (1) resource allocation to each client; (2) utility of individual clients; (3) server utility (exact in the oracle; optimistic estimate in the online scheme). The closer the approximate NE results given by the online scheme are to those given by the oracle, the better our online scheme converges.

As shown in Fig. 3 and Fig. 4, the results indicate that the online scheme's approximate NE progressively converges to the oracle's exact NE as the number of training rounds increases. Resource allocation, client utilities, and the server's optimistic utility estimates all become increasingly aligned with the oracle results. This validates the efficacy of the online scheme in approximating the optimal offline solution over time. Additionally, the optimistic estimates of the server utility consistently exceed the actual utility values, thereby confirming the validity of Theorem 3.

D. Long-term Performance

We evaluate the long-term effectiveness of our proposed online scheme by analyzing its regret performance. We perform evaluations on both synthetic and real-world (Yahoo) datasets, and the results are shown in Fig. 5 and Fig. 6, respectively. According to Theorem 4, the regret bound depends on the number of clients N and the parameter γ. Hence, we analyze how varying N and γ affects regret performance.

As shown in Fig. 5a and Fig. 6a, the regret increases with N, but not linearly. This is because the theoretical regret bound is derived by relaxing the actual number of selected clients and the number of times each client is selected. Due to the dynamic nature of service quality, the number of selected clients per round varies and is difficult to bound precisely. Thus, we conservatively bound the regret using the total number of clients N and total rounds T, which results in a loose dependency on N in the regret bound. As shown in Fig. 5b and Fig. 6b, regret increases linearly with γ. This aligns with Theorem 4, as γ represents the ratio between virtual task execution performance and actual monetary payment. A higher γ amplifies the regret bound, reflecting increased sensitivity to errors in task performance estimation.

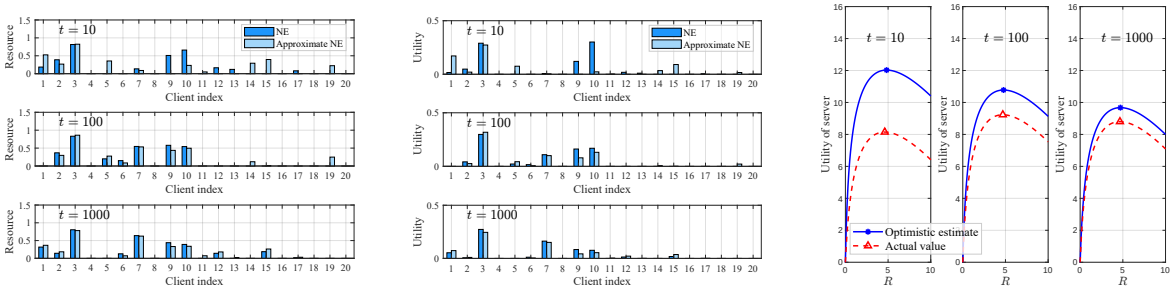
To further evaluate the robustness of our approach, we compare the proposed online scheme against two baselines:

- OIC without UCB (OIC w/o UCB): This variant uses ridge regression for service quality estimation but does not incorporate exploration, limiting its ability to identify high-performing clients.
- ϵ-greedy: This strategy explores with probability ϵ by randomly assigning tasks to all clients, and exploits the estimated best task otherwise. We set $\epsilon = \min 1, \frac{K}{t}$ following [31] to ensure sublinear regret.

As shown in Fig. 7, OIC outperforms both baselines. OIC w/o UCB performs the worst due to the absence of exploration, which leads to the exclusion of potentially high-quality clients based on initial poor performance. The ϵ-greedy approach performs better by incorporating exploration, but its naive strategy incurs linear regret increase during some exploratory rounds. In contrast, OIC achieves a more efficient exploration-exploitation tradeoff, resulting in consistently lower regret.

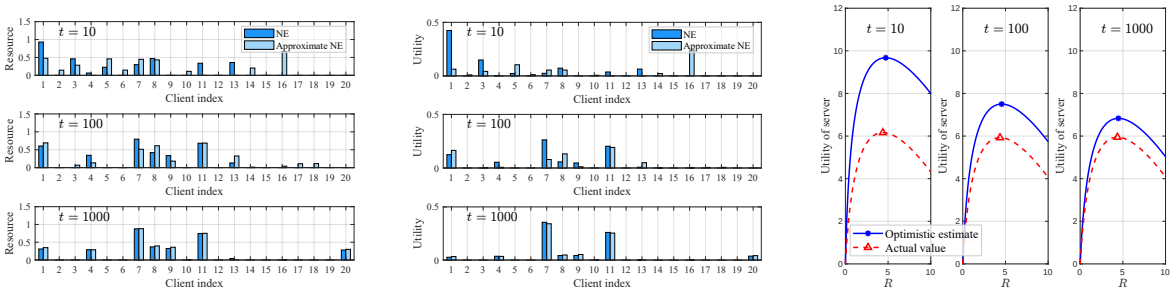
VI. CONCLUSION

In this paper, we propose a novel online incentivized and context-aware scheme, OIC, for task assignment in environments with unknown, context-dependent service quality.



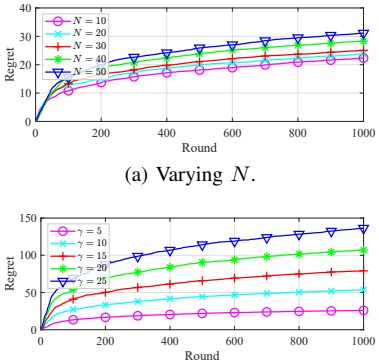
(a) Resource to be spent. (b) Client's utility. (c) Server's utility.

Fig. 3. Gap between online strategy and offline strategy (synthetic dataset).



(a) Resource to be spent. (b) Client's utility. (c) Server's utility.

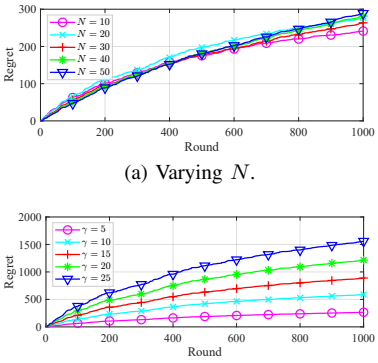
Fig. 4. Gap between online strategy and offline strategy (Yahoo dataset).



(a) Varying N.

(b) Varying γ.

Fig. 5. Regret performance (synthetic dataset).



(a) Varying N.

(b) Varying γ.

Fig. 6. Regret performance (Yahoo dataset).

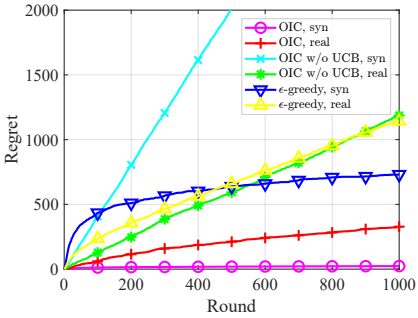


Fig. 7. Regret comparison.

Assuming that service quality follows a linear function of dynamic task contexts, we apply online learning to approximate the optimal task assignment strategy. We quantify uncertainty by deriving an upper confidence bound on the server's utility and establishing a sublinear regret bound, thereby guaranteeing the long-term performance of OIC. Comprehensive experiments on both synthetic and real-world datasets demonstrate the effectiveness of OIC in incentivizing client participation and maximizing the server's expected utility over time.

References

[1] J. Langford and T. Zhang, "The epoch-greedy algorithm for multi-armed bandits with side information," *Advances in neural information processing systems*, vol. 20, 2007.

[2] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proceedings of the 19th international conference on World wide web*, 2010, pp. 661–670.

[3] W. Chu, L. Li, L. Reyzin, and R. Schapire, "Contextual bandits with linear payoff functions," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 208–214.

[4] S. Agrawal and N. Goyal, "Thompson sampling for contextual bandits with linear payoffs," in *International conference on machine learning*. PMLR, 2013, pp. 127–135.

[5] A. Slivkins, "Contextual bandits with similarity information," in *Proceedings of the 24th annual Conference On Learning Theory*. JMLR Workshop and Conference Proceedings, 2011, pp. 679–702.

[6] N. An, R. Wang, Z. Luan, D. Qian, J. Cai, and H. Zhang, "Adaptive assignment for quality-aware mobile sensing network with strategic users," in *2015 IEEE 17th International Conference on High Performance Computing and Communications, 2015 IEEE 7th International Symposium on Cyberspace Safety and Security, and 2015 IEEE 12th*

International Conference on Embedded Software and Systems. IEEE, 2015, pp. 541–546.

[7] H. Zhang, Y. Ma, and M. Sugiyama, "Bandit-based task assignment for heterogeneous crowdsourcing," *Neural computation*, vol. 27, no. 11, pp. 2447–2475, 2015.

[8] S. Elahi, A. Nika, and C. Tekin, "Online context-aware task assignment in mobile crowdsourcing via adaptive discretization," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 1, pp. 305–320, 2022.

[9] S. K. née Müller, C. Tekin, M. van der Schaar, and A. Klein, "Context-aware hierarchical online learning for performance maximization in mobile crowdsourcing," *IEEE/ACM Transactions on Networking*, vol. 26, no. 3, pp. 1334–1347, 2018.

[10] D. Yang, G. Xue, X. Fang, and J. Tang, "Incentive mechanisms for crowdsensing: Crowdsourcing with smartphones," *IEEE/ACM transactions on networking*, vol. 24, no. 3, pp. 1732–1744, 2015.

[11] Z. Cai, Z. Duan, and W. Li, "Exploiting multi-dimensional task diversity in distributed auctions for mobile crowdsensing," *IEEE Transactions on Mobile Computing*, vol. 20, no. 8, pp. 2576–2591, 2020.

[12] Y. Li, F. Li, S. Yang, P. Zhou, L. Zhu, and Y. Wang, "Three-stage stackelberg long-term incentive mechanism and monetization for mobile crowdsensing: An online learning approach," *IEEE Transactions on Network Science and Engineering*, vol. 8, no. 2, pp. 1385–1398, 2021.

[13] Q. Xu, Z. Su, D. Fang, and Y. Wu, "BASIC: Distributed task assignment with auction incentive in uav-enabled crowdsensing system," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 2, pp. 2416–2430, 2023.

[14] Y. Wang, J. Jiang, and T. Mu, "Context-aware and energy-driven route optimization for fully electric vehicles via crowdsourcing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1331–1345, 2013.

[15] B. Gu, Y. Chen, H. Liao, Z. Zhou, and D. Zhang, "A distributed and context-aware task assignment mechanism for collaborative mobile edge computing," *Sensors*, vol. 18, no. 8, p. 2423, 2018.

[16] M. C. Pestana and V. Vieira, "Context-aware task distribution for mobile crowdsourcing," in *Proceedings of the 17th Brazilian Symposium on Human Factors in Computing Systems*, 2018, pp. 1–8.

[17] Y. Zhao, K. Zheng, H. Yin, G. Liu, J. Fang, and X. Zhou, "Preference-aware task assignment in spatial crowdsourcing: from individuals to groups," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 7, pp. 3461–3477, 2020.

[18] J. Wang, Y. Yang, S. Wang, C. Chen, D. Wang, and Q. Wang, "Context-aware personalized crowdtesting task recommendation," *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 3131–3144, 2021.

[19] L. Kazemi and C. Shahabi, "Geocrowd: enabling query answering with spatial crowdsourcing," in *Proceedings of the 20th international conference on advances in geographic information systems*, 2012, pp. 189–198.

[20] B. Zhao, Y. Wang, Y. Li, Y. Gao, and X. Tong, "Task allocation model based on worker friend relationship for mobile crowdsourcing," *Sensors*, vol. 19, no. 4, p. 921, 2019.

[21] D. Deng, C. Shahabi, and U. Demiryurek, "Maximizing the number of worker's self-selected tasks in spatial crowdsourcing," in *Proceedings of the 21st acm sigspatial international conference on advances in geographic information systems*, 2013, pp. 324–333.

[22] M. H. Cheung, F. Hou, J. Huang, and R. Southwell, "Distributed time-sensitive task selection in mobile crowdsensing," *IEEE Transactions on Mobile Computing*, vol. 20, no. 6, pp. 2172–2185, 2020.

[23] Z. Wang, L. Gao, and J. Huang, "Socially-optimal mechanism design for incentivized online learning," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1828–1837.

[24] Y. Zhao, K. Zheng, J. Guo, B. Yang, T. B. Pedersen, and C. S. Jensen, "Fairness-aware task assignment in spatial crowdsourcing: Game-theoretic approaches," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 265–276.

[25] Z. Feng, Y. Zhu, Q. Zhang, L. M. Ni, and A. V. Vasilakos, "Trac: Truthful auction for location-aware collaborative sensing in mobile crowdsourcing," in *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*. IEEE, 2014, pp. 1231–1239.

[26] X. Zhang, G. Xue, R. Yu, D. Yang, and J. Tang, "Truthful incentive mechanisms for crowdsourcing," in *2015 IEEE Conference on Computer Communications (INFOCOM)*. IEEE, 2015, pp. 2830–2838.

[27] X. Wang, Y. Zhao, C. Qiu, Z. Liu, J. Nie, and V. C. Leung, "Infedge: A blockchain-based incentive mechanism in hierarchical federated learning for end-edge-cloud communications," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 12, pp. 3325–3342, 2022.

[28] J. Ji, K. Cai, Z. Chen, and J. Zhang, "OIC: an online incentivized and context-aware scheme for task assignment: Technical report," 2025. [Online]. Available: https://kechao.me/pubs/TR/TR-360-video-online-caching.pdf

[29] Y. Abbasi-Yadkori, D. Pál, and C. Szepesvári, "Improved algorithms for linear stochastic bandits," *Advances in neural information processing systems*, vol. 24, 2011.

[30] O. Chapelle and Y. Chang, "Yahoo! learning to rank challenge overview," in *Proceedings of the learning to rank challenge*. PMLR, 2011, pp. 1–24.

[31] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine learning*, vol. 47, pp. 235–256, 2002.

Appendix A
Proofs of Theorem 1

Proof. **Proof of (1)**: We prove this statement by contradiction. Suppose that $|\tilde{\mathcal{S}}_t^k| = 0$, then client i can increase its utility from 0 to $\frac{P_t^k}{2}$ by unilaterally changing its computational resource from 0 to $\frac{P_t^k}{2c_i}$, contradicting the NE assumption. This proves that $|\tilde{\mathcal{S}}_t^k| \geq 1$. Suppose that $|\tilde{\mathcal{S}}_t^k| = 1$. This means $\tilde{r}_{i,t}^k > 0$ for some $i \in \mathcal{N}$, and $\tilde{r}_{j,t}^k = 0$ for all $j \in \mathcal{N} \setminus \{i\}$. The current utility of client i is $P_t^k - \tilde{r}_{i,t}^k c_i$. Client i can increase its utility by unilaterally changing its sensing time from $\tilde{r}_{i,t}^k$ to $\frac{\tilde{r}_{i,t}^k}{2}$, again contradicting the NE assumption. Therefore $|\tilde{\mathcal{S}}_t^k| \geq 2$.

Proof of (2): The optimal strategy can be obtained by letting the derivative of the utility of client be 0, i.e.,

$$\begin{aligned}\frac{\partial u_{i,t}^k}{\partial r_{i,t}^k} &= \frac{q_{i,t}^k}{\sum_{j\in\mathcal{N}} q_{j,t}^k r_{j,t}^k} P_t^k - \frac{q_{i,t}^k}{(\sum_{j\in\mathcal{N}} q_{j,t}^k r_{j,t}^k)^2} P_t^k - c_i \\ &= \frac{q_{i,t}^k}{\sum_{j\in\tilde{\mathcal{S}}_t^k} q_{j,t}^k r_{j,t}^k} P_t^k - \frac{q_{i,t}^k}{(\sum_{j\in\tilde{\mathcal{S}}_t^k} q_{j,t}^k r_{j,t}^k)^2} P_t^k - c_i = 0.\end{aligned} \tag{14}$$

Summing up over the clients in $\tilde{\mathcal{S}_t^k}$, we have

$$n_0 P_t^k - P_t^k = \sum_{j\in\tilde{\mathcal{S}}_t^k} q_{j,t}^k r_{j,t}^k \sum_{j\in\tilde{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}, \tag{15}$$

thus we have

$$\sum_{j\in\tilde{\mathcal{S}}_t^k} q_{j,t}^k r_{j,t}^k = \frac{(n_0-1)P_t^k}{\sum_{j\in\tilde{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}. \tag{16}$$

Substituting (16) into (14) and considering $\tilde{r}_{i,t}^k = 0$ for any $j \in \mathcal{N} \setminus \{\tilde{\mathcal{S}_t^k}\}$, we obtain the optimal strategy:

$$\tilde{r}_{i,t}^k = \frac{(n_0-1)P_t^k}{q_{i,t}^k \sum_{j\in\tilde{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}} \left(1 - \frac{(n_0-1)c_i}{q_{i,t}^k \sum_{j\in\tilde{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}\right). \tag{17}$$

Therefore, the statement (2) is proved.

Proof of (3): We prove this statement by contradiction. By definition of $\tilde{\mathcal{S}}_t^k$, we know that $\tilde{r}_{i,t}^k > 0$ for every $i \in \tilde{\mathcal{S}}_t^k$. From statement (2), $\tilde{r}_{i,t}^k > 0$ implies $\frac{(n_0-1)c_i}{q_{i,t}^k \sum_{j\in\tilde{\mathcal{S}}_t^k q_{j,t}^k}} < 1$, i.e., $\forall i \in \tilde{\mathcal{S}}_t^k$, $\frac{c_i}{q_{i,t}^k} < \frac{\sum_{j\in\tilde{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}{n_0-1}$. Therefore we have $\max_{j\in\tilde{\mathcal{S}}_t^k}\{\frac{c_j}{q_{j,t}^k}\} <$

$\frac{\sum_{j\in\tilde{S}_t^k}\frac{c_j}{q_{j,t}^k}}{n_0-1}$. Assume that $\frac{c_i}{q_{i,t}^k} \le \max_{j\in\tilde{S}_t^k}\{\frac{c_j}{q_{j,t}^k}\} < \frac{\sum_{j\in\tilde{S}_t^k}\frac{c_j}{q_{j,t}^k}}{n_0-1}$ but $i \notin \tilde{S}_t^k$, i.e., $\tilde{r}_{i,t}^k = 0$.

$$\begin{aligned}\frac{\partial u_{i,t}^k}{\partial r_{i,t}^k} &= \frac{q_{i,t}^k \sum_{j\in N\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{(\sum_{j\in N} q_{j,t}^k r_{j,t}^k)^2} P_t^k - c_i \\ &= \frac{q_{i,t}^k \sum_{j\in\tilde{S}_t^k} q_{j,t}^k r_{j,t}^k}{(\sum_{j\in\tilde{S}_t^k} q_{j,t}^k r_{j,t}^k)^2} P_t^k - c_i \\ &= \frac{q_{i,t}^k}{\sum_{j\in\tilde{S}_t^k} q_{j,t}^k r_{j,t}^k} P_t^k - c_i \\ &= \frac{q_{i,t}^k \sum_{j\in\tilde{S}_t^k}\frac{c_j}{q_{j,t}^k}}{n_0-1} - c_i \\ &> q_{i,t}^k \max_{j\in\tilde{S}_t^k}\{\frac{c_j}{q_{j,t}^k}\} - c_i \ge 0,\end{aligned}$$

where the third equation comes from (16). This means that client i can increase its utility by unilaterally increasing its sensing time from $\tilde{r}_{i,t}^k = 0$, contradicting the NE assumption.

Proof of (4): Statements (1) and (3) imply that $\tilde{S}_t^k = \{i^{(1)}, i^{(2)}, ..., i^{(g)}\}$ for some integer g in $[2, N]$. From $\frac{c_{i^{(g)}}}{q_{i^{(g)},t}^k} < \frac{\sum_{j\in\tilde{S}_t^k}\frac{c_j}{q_{j,t}^k}}{n_0-1}$, we have $g \le h$. Assume that $g < h$, then $\frac{c_{i^{(g+1)}}}{q_{i^{(g+1)},t}^k} \ge \frac{\sum_{j=1}^{g+1}\frac{c_{i^{(j)}}}{q_{i^{(j)},t}^k}}{g}$, which implies $\sum_{j=1}^{g}\frac{c_{i_j^{Sort}}}{q_{i_j^{Sort},t}^k}\frac{c_{i^{(g+1)}}}{q_{i^{(g+1)},t}^k} > 0$.

$$\begin{aligned}\frac{\partial u_{i^{(g+1)},t}^k}{\partial r_{i^{(g+1)},t}^k} &= \frac{q_{i^{(g+1)},t}^k \sum_{j\in\tilde{S}_t^k} q_{j,t}^k r_{j,t}^k}{(\sum_{j\in\tilde{S}_t^k} q_{j,t}^k r_{j,t}^k)^2} P_t^k - c_{i^{(g+1)}} \\ &= \frac{q_{i^{(g+1)},t}^k}{\sum_{j\in\tilde{S}_t^k} q_{j,t}^k r_{j,t}^k} P_t^k - c_{i^{(g+1)}} \\ &= \frac{q_{i_{g+1}^{sort},t}^k \sum_{j=1}^{g} q_{i^{(j)},t}^k}{g-1} - c_{i^{(g+1)}} > 0,\end{aligned}$$

where the third equation comes from (16). This contradiction proves $g = h$, and thus proves statement (4).

Appendix B
Proof of Theorem 2

Proof. The following three observations are used to prove Algorithm 1 finds the Nash equilibrium of task k.

(1) For any $i \in \mathcal{S}_t^k$, $\frac{c_i}{q_{i,t}^k} \ge \frac{\Sigma_{j\in\mathcal{S}_t^k}\frac{c_j}{q_{j,t}^k}}{n_0-1}$, where $n_0 = |\mathcal{S}_t^k|$.

(2) $\sum_{j\in S_t^k} q_{j,t}^k r_{j,t}^k{}^{\text{NE}} = \frac{(n_0-1)P_t^k}{\Sigma_{j\in S_t^k}\frac{c_j}{q_{j,t}^k}}$.

(3) $\sum_{j\in\mathcal{S}_t^k\setminus\{i\}} q_{j,t}^k r_{j,t}^k{}^{\text{NE}} = \frac{(n_0-1)^2 c_i P_t^k}{q_{j,t}^k(\Sigma_{j\in S_t^k}\frac{c_j}{q_{j,t}^k})^2}$

Based on the above observations, we prove two cases respectively: client $i \in \mathcal{S}_t^k$ and client $i \notin \mathcal{S}_t^k$. For any $i \notin \mathcal{S}_t^k$,

$$\begin{aligned}\frac{c_i}{q_{i,t}^k}\sum_{j\in\mathcal{S}_t^k} q_{j,t}^k r_{j,t}^k{}^{\text{NE}} &= \frac{c_i}{q_{i,t}^k}\sum_{j\in\mathcal{S}_t^k} q_{j,t}^k r_{j,t}^k{}^{\text{NE}} \\ &\overset{(2)}{=} \frac{c_i}{q_{i,t}^k}\frac{(n_0-1)P_t^k}{\sum_{j\in\mathcal{S}_t^k}\frac{c_j}{q_{j,t}^k}} \\ &\overset{(1)}{\ge} \frac{\sum_{j\in\mathcal{S}_t^k}\frac{c_j}{q_{j,t}^k}}{n_0-1}\frac{(n_0-1)P_t^k}{\sum_{j\in\mathcal{S}_t^k}\frac{c_j}{q_{j,t}^k}} = P_t^k, \quad (18)\end{aligned}$$

thus $\beta_{i,t}^k(r_{-i,t}^k{}^{\text{NE}}) = 0$. For any $i \in \mathcal{S}_t^k$,

$$\sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k{}^{\text{NE}} = \sum_{j\in\mathcal{S}_t^k\setminus\{i\}} q_{j,t}^k r_{j,t}^k{}^{\text{NE}} \quad (19)$$

$$\overset{(3)}{=} \frac{c_i}{q_{i,t}^k}\frac{(n_0-1)^2 c_i P_t^k}{q_{j,t}^k(\Sigma_{j\in\mathcal{S}_t^k}\frac{c_j}{q_{j,t}^k})^2} = (\frac{c_i}{q_{i,t}^k})^2\frac{(n_0-1)^2 P_t^k}{(\Sigma_{j\in\mathcal{S}_t^k}\frac{c_j}{q_{j,t}^k})^2}, \quad (20)$$

$$\begin{aligned}(n_0-1)\frac{c_i}{q_{i,t}^k} &= (i-1)\frac{c_i}{q_{i,t}^k} + (n_0-i)\frac{c_i}{q_{i,t}^k} \\ &< \sum_{j=1}^{i}\frac{c_i}{q_{j,t}^k} + \sum_{j=i+1}^{n_0}\frac{c_i}{q_{j,t}^k} = \sum_{j=1}^{n_0}\frac{c_i}{q_{j,t}^k}, \quad (21)\end{aligned}$$

where the second inequality holds because $\frac{c_i}{q_{i,t}^k} < \frac{\frac{c_i}{q_{i,t}^k}+\sum_{j\in S_t^k}\frac{c_j}{q_{j,t}^k}}{|S_t^k|}$ holds for client $i \in \mathcal{S}_t^k$, which comes from Line 4 of Algorithm 1. Therefore, we have

$$\begin{aligned}&\frac{c_i}{\hat{q}_{i,t}^k}\sum_{j\in\mathcal{N}\setminus\{i\}} \hat{q}_{j,t}^k r_{j,t}^k{}^{\text{NE}} \\ &< (\frac{\sum_{j=1}^{n_0}\frac{c_j}{\hat{q}_{j,t}^k}}{n_0-1})^2\frac{(n_0-1)^2 P_t^k}{(\Sigma_{j\in S_t^k}\frac{c_j}{q_{j,t}^k})^2} = P_t^k, \quad (22)\end{aligned}$$

and thus the strategy of client $i \in \mathcal{S}_t^k$ is

$$\begin{aligned}\beta_{i,t}^k(r_{-i,t}^k{}^{\text{NE}}) &= \frac{\sqrt{\frac{q_{i,t}^k P_t^k \sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{c_i}} - \sum_{j\in\mathcal{N}\setminus\{i\}} q_{j,t}^k r_{j,t}^k}{q_{i,t}^k} \\ &= \frac{(|S_t^k|-1)P_t^k}{q_{i,t}^k\sum_{j\in S_t^k}\frac{c_j}{q_{j,t}^k}}(1 - \frac{(|S_t^k|-1)c_i}{q_{i,t}^k\sum_{j\in S_t^k}\frac{c_j}{q_{j,t}^k}}) = r_{i,t}^k{}^{\text{NE}}. \quad (23)\end{aligned}$$

Therefore, the strategy in Algorithm 1 is the NE of the Stackelberg game of task k, which corresponds to the solution in (6). □

□

Appendix C
Proofs of Theorem 3

In this section, we provide the proof of Theorem 4.

Lemma 1. *Suppose that there are three sequences:*

1. $\{a_1, a_2, \cdots, a_n\}$, *where* $a_1 < a_2 < \cdots < a_n$,
2. $\{b_1, b_2, \cdots, b_n\}$, *where* $b_1 \le a_1$, $b_2 \le a_2$, $\cdots$, $b_n \le a_n$,

3. *Sort sequence 2 in ascending order and denoted as* $\{b'_1, b'_2, \cdots, b'_n\}$, *where* $b'_1 < b'_2 < \cdots < b'_n$.

For sequence 1 and sequence 3, we select elements according to the following rules:

- *First select the two smallest elements in the sequence 1 and 3 and denote the selected sets of sequence 1 and 3 as* $\mathcal{S}_a$ *and* $\mathcal{S}_b$ *respectively (note that sequences 1 and 3 are sorted in ascending order, so the first two elements are the two smallest elements).*
- *Then for the elements in sequences 1, we verify in order if* $a_i < \frac{a_i + \sum_{j\in\mathcal{S}_a} a_j}{|\mathcal{S}_a|}$. *If the condition is satisfied, then* a_i *will be selected into* $\mathcal{S}_a$ *and* $\mathcal{S}_a$ *will be updated. If not, then the selection stops.*
- *The same rule* $b'_h < \frac{b'_h + \sum_{j\in\mathcal{S}_b} b'_j}{|\mathcal{S}_b|}$ *is used to determine whether to select* b'_h *and update* $\mathcal{S}_b$.

Finally, we denote the size of final result of $\mathcal{S}_a$ *and* $\mathcal{S}_b$ *as* $n_a = |\mathcal{S}_a|$ *and* $n_b = |\mathcal{S}_b|$ *respectively. We have* $\frac{\sum_{j=1}^{n_a} a_j}{n_a - 1} \geq \frac{\sum_{j=1}^{n_b} b'_j}{n_b - 1}$.

Proof. We prove this lemma in three cases: (1) $n_a = n_b$, (2) $n_a < n_b$, and (3) $n_a > n_b$.

We first prove the case of $n_a = n_b$. Obviously, we have

$$\frac{\sum_{j=1}^{n_b} b'_j}{n_b - 1} \leq \frac{\sum_{j=1}^{n_b} b_j}{n_b - 1} \leq \frac{\sum_{j=1}^{n_b} a_j}{n_b - 1} = \frac{\sum_{j=1}^{n_a} a_j}{n_a - 1}.$$

Then we prove the case of $n_a < n_b$. For any $h < n_b$, we have $b'_h < \frac{\sum_{j=1}^{h-1} b'_j}{h-2}$ from the selection condition $b'_h < \frac{b'_h + \sum_{j\in\mathcal{S}_b} b'_j}{|\mathcal{S}_b|}$. Thus we have

$$\begin{aligned}
\frac{\sum_{j=1}^{n_b} b'_j}{n_b - 1} &= \frac{b'_{n_b} + \sum_{j=1}^{n_b-1} b'_j}{n_b - 1} < \frac{\frac{\sum_{j=1}^{n_b-1} b'_j}{n_b-2} + \sum_{j=1}^{n_b-1} b'_j}{n_b - 1} \\
&= \frac{\sum_{j=1}^{n_b-1} b'_j}{n_b - 2} = \frac{b'_{n_b-1} + \sum_{j=1}^{n_b-2} b'_j}{n_b - 2} \\
&< \frac{\frac{\sum_{j=1}^{n_b-2} b'_j}{n_b-3} + \sum_{j=1}^{n_b-2} b'_j}{n_b - 2} = \frac{\sum_{j=1}^{n_b-2} b'_j}{n_b - 3} \\
&< \cdots < \frac{\sum_{j=1}^{n_b-\Lambda n} b'_j}{n_b - \Delta n - 1} = \frac{\sum_{j=1}^{n_a} b'_j}{n_a - 1} \\
&\leq \frac{\sum_{j=1}^{n_a} b_j}{n_a - 1} \leq \frac{\sum_{j=1}^{n_a} a_j}{n_a - 1}.
\end{aligned} \tag{24}$$

Finally, we prove the case of $n_a > n_b$. For any $i \leq n_a$, we have $a_i \leq a_{n_a} < \frac{a_{n_a} + \sum_{j=1}^{n_a-1} a_j}{n_a - 1} < \frac{\sum_{j=1}^{n_a} a_j}{n_a - 1}$ from the selection condition $a_i < \frac{a_i + \sum_{j\in\mathcal{S}_a} a_j}{|\mathcal{S}_a|}$. For any $h > n_b$, we have $b'_h \geq \frac{\sum_{j=1}^{n_b} b'_j}{n_b - 1}$ since it does not satisfy the selection condition $b'_h < \frac{b'_h + \sum_{j\in\mathcal{S}_b} b'_j}{|\mathcal{S}_b|}$. Since $n_a > n_b$, there must be at least one element in sequence 1 which is selected into $\mathcal{S}_a$, but its corresponding lower bound in sequence 3 is not selected into $\mathcal{S}_b$ Denote any of these special elements in sequence 1 as a_{i_0}, and denote its corresponding lower bound in sequence 2 and sequence 3 as b_{i_0} and b'_{h_0}, we have $a_{i_0} \geq b_{i_0} = b'_{h_0}$. With the help of this special element, we have $\frac{\sum_{j=1}^{n_b} b'_j}{n_b - 1} \leq b'_{h_0} \leq a_{i_0} < \frac{\sum_{j=1}^{n_a} a_j}{n_a - 1}$. □

Proof.

$$u_{0,t}^k = \gamma \log(\gamma X_t^k) + \frac{1}{X_t^k} - \gamma, X_t^k = \frac{n_0 - 1}{\sum_{j\in\mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}} \tag{25}$$

$$\hat{u}_{0,t}^k = \gamma \log(\gamma \hat{X}_t^k) + \frac{1}{\hat{X}_t^k} - \gamma, \hat{X}_t^k = \frac{\hat{n}_0 - 1}{\sum_{j\in\hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}. \tag{26}$$

For function $f(x) = \gamma \log(\gamma x) + \frac{1}{x} - \gamma$, we compute the first derivative of $f(x)$ as $\frac{\partial f(x)}{\partial x} = \frac{\gamma}{x} - \frac{1}{x^2}$. Let $\frac{\partial f(x)}{\partial x} = 0$, we have $x = \frac{1}{\gamma}$. Since $\frac{\partial f(x)}{\partial x}|_{x=\frac{2}{\gamma}} = \frac{\gamma^2}{4} > 0$ and $\frac{\partial f(x)}{\partial x}|_{x=\frac{1}{2\gamma}} = -2\gamma^2 < 0$, we have $f(x)$ is a convex function and $f(x)$ reaches its minimum at $x = \frac{1}{\gamma}$. Therefore, for $x_1 \geq x_2 \geq \frac{1}{\gamma}$, we have $f(x_1) \geq f(x_2)$. Therefore, proving $\hat{u}_{0,t}^k \geq u_{0,t}^k$ is equivalent to proving $\hat{X}_t^k \geq X_t^k$.

We utilizes Lemma 1 to prove $\hat{X}_t^k \geq X_t^k$, where sequence 1 can be referred to as the sorted sequence $\{\frac{c_1}{q_{1,t}^k}, \frac{c_2}{q_{2,t}^k}, \cdots, \frac{c_i}{q_{i,t}^k}\}$, where $\frac{c_1}{q_{1,t}^k} \leq \frac{c_2}{q_{2,t}^k} \leq \cdots \leq \frac{c_i}{q_{i,t}^k}$. Sequence 2 can be referred to as $\{\frac{c_1}{\hat{q}_{1,t}^k}, \frac{c_2}{\hat{q}_{2,t}^k}, \cdots, \frac{c_i}{\hat{q}_{i,t}^k}\}$ and $\frac{c_1}{\hat{q}_{1,t}^k} \leq \frac{c_1}{q_{1,t}^k}, \frac{c_2}{\hat{q}_{2,t}^k} \leq \frac{c_1}{q_{1,t}^k}, \cdots, \frac{c_i}{\hat{q}_{i,t}^k} \leq \frac{c_1}{q_{1,t}^k}$ with probability at least $(1-\delta)^N$. Sequence 3 can be referred to as an ascending sort of $\{\frac{c_1}{\hat{q}_{1,t}^k}, \frac{c_2}{\hat{q}_{2,t}^k}, \cdots, \frac{c_i}{\hat{q}_{i,t}^k}\}$.

According to Lemma 1, we have

$$\frac{\sum_{j\in\hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}{\hat{n}_0 - 1} \leq \frac{\sum_{j\in\mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}{n_0 - 1}. \tag{27}$$

Since $X_t^k = \frac{n_0 - 1}{\sum_{j\in\mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}$ and $\hat{X}_t^k = \frac{\hat{n}_0 - 1}{\sum_{j\in\hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}$, we have $\hat{X}_t^k \geq X_t^k$ with probability at least $(1-\delta)^N$. Thus we prove Lemma 3. □

Appendix D
Proofs of Theorem 4

In this section, we provide the proof of Theorem 4. First, we bound the **single-round regret**. Specifically, we show in Lemma 2 that the regret in the server's utility in a given round can be bounded by the uncertainty—i.e., the confidence bounds—of the clients' utility estimates in that round. This result establishes a crucial connection between the server's performance and the quality of client-side task assignments.

Lemma 2. *For a single round t, the regret of Algorithm 2 is upper bounded by*

$$r_t \leq \gamma \log(1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j\in\hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}), \tag{28}$$

where $q_{\min}$ *denotes the minimum of data quality.*

Proof.

$$\begin{aligned}
r_t &= u_{0,t}^{k^*} - u_{0,t}^{k_t} \\
&= (u_{0,t}^{k^*} - \hat{u}_{0,t}^{k^*}) + (\hat{u}_{0,t}^{k^*} - \hat{u}_{0,t}^{k_t}) + (\hat{u}_{0,t}^{k_t} - u_{0,t}^{k_t}) \\
&\leq \hat{u}_{0,t}^{k_t} - u_{0,t}^{k_t},
\end{aligned}$$

where the inequality is due to the fact that $\hat{u}_{0,t}^{k^*} \geq u_{0,t}^{k^*}$ according to Lemma 3 and $\hat{u}_{0,t}^{k_t} \geq u_{0,t}^{k_t}$ since the server selects the estimated optimal arm k_t.

Next, we analyze the difference between $\hat{u}_{0,t}^{k_t}$ and $u_{0,t}^{k_t}$. To simplify the notation, we analyze the difference between $\hat{u}_{0,t}^{k}$ and $u_{0,t}^{k}$ in the following steps, which is also the bound of the difference between $\hat{u}_{0,t}^{k_t}$ and $u_{0,t}^{k_t}$.

$$\begin{aligned}
&\hat{u}_{0,t}^k - u_{0,t}^k \\
&= \gamma \log(\gamma \hat{X}_t^k) + \frac{1}{\hat{X}_t^k} - \gamma - (\gamma \log(\gamma X_t^k) + \frac{1}{X_t^k} - \gamma) \\
&= \gamma \log(\frac{\gamma \hat{X}_t^k}{\gamma X_t^k}) + \frac{1}{\hat{X}_t^k} - \frac{1}{X_t^k} \\
&\leq \gamma \log(\frac{\hat{X}_t^k}{X_t^k}),
\end{aligned}$$

where the inequality is due to the fact that $\hat{X}_t^k \geq X_t^k$ we proved in Lemma 3. Therefore, we need to analyze the bound of $\frac{\hat{X}_t^k}{X_t^k}$. We analyze the bound from three cases: $\mathcal{S}_t^k = \hat{\mathcal{S}}_t^k$, $\mathcal{S}_t^k \subset \hat{\mathcal{S}}_t^k$, and $\mathcal{S}_t^k \not\subset \hat{\mathcal{S}}_t^k$.

For the case of $\mathcal{S}_t^k = \hat{\mathcal{S}}_t^k$, we have

$$\begin{aligned}
\frac{\hat{X}_t^k}{X_t^k} &= \frac{\frac{\hat{n}_0 - 1}{\sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}}}{\frac{n_0 - 1}{\sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}} = \frac{\sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}{\sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}} \\
&= \sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k \sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}} \leq \sum_{j \in \hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k \frac{c_j}{\hat{q}_{j,t}^k} (\hat{n}_0 - 1)} \\
&= \frac{1}{\hat{n}_0 - 1} \sum_{j \in \hat{\mathcal{S}}_t^k} \frac{\hat{q}_{j,t}^k}{q_{j,t}^k} = \frac{1}{\hat{n}_0 - 1} \sum_{j \in \hat{\mathcal{S}}_t^k} \frac{q_{j,t}^k + \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{j,t}^k} \\
&= \frac{1}{\hat{n}_0 - 1} (\hat{n}_0 + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}) \\
&\leq \frac{\hat{n}_0}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}} \\
&= 1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}.
\end{aligned}$$

For the case of $\hat{\mathcal{S}}_t^k \subset \mathcal{S}_t^k$, we obtain $\frac{\sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}{n_0 - 1} \leq \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}{\hat{n}_0 - 1}$ by using the same method as (24) when proving case (2) in proof of lemma 1. Thus we have

$$\begin{aligned}
\frac{\hat{X}_t^k}{X_t^k} &= \frac{\frac{\hat{n}_0 - 1}{\sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}}}{\frac{n_0 - 1}{\sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}} \leq \frac{\frac{\hat{n}_0 - 1}{\sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}}}{\frac{\hat{n}_0 - 1}{\sum_{j \in \hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}} = \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \frac{c_j}{q_{j,t}^k}}{\sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}} \\
&\leq 1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}.
\end{aligned}$$

For the case of $\hat{\mathcal{S}}_t^k \not\subset \mathcal{S}_t^k$, there must be at least one client that is selected into $\hat{\mathcal{S}}_t^k$, but is not selected into $\mathcal{S}_t^k$. Thus we can find the special client $j_0 = h_0$ that satisfies $j_0 \in \hat{\mathcal{S}}_t^k$ and $h_0 \notin \mathcal{S}_t^k$. We have $\frac{c_{j_0}}{\hat{q}_{j_0,t}^k} < \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \frac{c_j}{\hat{q}_{j,t}^k}}{\hat{n}_0 - 1}$ and $\frac{c_{h_0}}{q_{h_0,t}^k} \geq \frac{\sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}{n_0 - 1}$ from the selection condition. Therefore, we have

$$\begin{aligned}
\frac{\hat{X}_t^k}{X_t^k} &= \frac{\frac{\hat{n}_0 - 1}{\sum_{h \in \hat{\mathcal{S}}_t^k} \frac{c_h}{\hat{q}_{h,t}^k}}}{\frac{n_0 - 1}{\sum_{j \in \mathcal{S}_t^k} \frac{c_j}{q_{j,t}^k}}} \leq \frac{\frac{\hat{q}_{j_0,t}^k}{c_{j_0}}}{\frac{q_{h_0,t}^k}{c_{h_0}}} = \frac{\frac{\hat{q}_{j_0,t}^k}{c_{j_0}}}{\frac{q_{j_0,t}^k}{c_{j_0}}} = \frac{\hat{q}_{j_0,t}^k}{q_{j_0,t}^k} \\
&= 1 + \frac{\alpha_{j_0,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j_0,t-1}^{-1}}}{q_{\min}} \\
&\leq 1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}.
\end{aligned}$$

Therefore, the regret in single round t is bounded by

$$\begin{aligned}
r_t &\leq \gamma \log(\frac{\hat{X}_t^k}{X_t^k}) \\
&\leq \gamma \log(1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}).
\end{aligned}$$

□

Next, we bound the **cumulative regret** of the server's utility. This is done by leveraging the confidence bounds on service quality to construct a martingale sequence, which enables us to show that the sum of the confidence bounds over time converges, as shown in Lemma 3.

Lemma 3 (Lemma 11 in [29])**.** *Let $\boldsymbol{x}_1, \boldsymbol{x}_2, ..., \boldsymbol{x}_n \in \mathbb{R}^d$ be a sequence in $\mathbb{R}^d$, $\boldsymbol{U}_0 \in \mathbb{R}^{d \times d}$ a positive defnite matrix and define $\boldsymbol{U}_n = \boldsymbol{U}_0 + \sum_{t=1}^n \boldsymbol{x}_t \boldsymbol{x}_t^\top$ for all t.Then, we have that*

$$\log\left(\frac{\det(\boldsymbol{U}_n)}{\det(\boldsymbol{U}_0)}\right) \leq \sum_{t=1}^n ||\boldsymbol{x}_t||_{\boldsymbol{U}_{t-1}^{-1}}^2$$

Further, if $||\boldsymbol{x}_t||_2 \leq L$, then

$$\sum_{t=1}^n \min\{1, ||\boldsymbol{x}_t||_{\boldsymbol{U}_{t-1}^{-1}}^2\} \leq 2(\log\det(\boldsymbol{U}_t) - \log\det \boldsymbol{U}_0),$$

and finally, if $\lambda_{\min}(\boldsymbol{U}_0) \geq \max(1, L^2)$ then

$$\sum_{t=1}^n ||\boldsymbol{x}_t||_{\boldsymbol{U}_{t-1}^{-1}}^2 \leq 2 \log \frac{\det(\boldsymbol{U}_n)}{\det(\boldsymbol{U}_0)}.$$

With the help of Lemma 2 and 3, we can analyze the regret of Algorithm 2 in multiple rounds.

Proof. According to Lemma 2, the regret in single round t is bounded by

$$r_t \leq \gamma \log(1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}). \quad (29)$$

Therefore, the total regret in T rounds is bounded by

$$\begin{aligned}
R_T &= \sum_{t=1}^T r_t \\
&\leq \sum_{t=1}^T \gamma \log(1 + \frac{1}{\hat{n}_0 - 1} + \frac{\sum_{j \in \hat{\mathcal{S}}_t^k} \alpha_{j,t} ||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}})
\end{aligned}$$

Denote $Y_t^k = \frac{\sum_{j\in\hat{\mathcal{S}}_t^k}\alpha_{j,t}||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}}$. Since $\frac{1}{\hat{n}_0-1}$ is small enough, it is resonable to assume that Y_t^k is large enough so that $1+\frac{1}{\hat{n}_0-1} \leq e^{Y_t^k} - Y_t^k$. Therefore, we can safely assume that $\log(1+\frac{1}{\hat{n}_0-1}+Y_t^k) \leq Y_t^k$. Thus, the total regret is bounded by

$$\begin{aligned}
R_T \leq & \gamma \sum_{t=1}^{T} \frac{\sum_{j\in\hat{\mathcal{S}}_t^k}\alpha_{j,t}||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}}}{q_{\min}} \\
= & \frac{\gamma}{q_{\min}} \sum_{j\in\mathcal{N}} \sum_{t\in\mathcal{T}_j} \alpha_{j,t}||\boldsymbol{x}_t||_{\boldsymbol{V}_{j,t-1}^{-1}} \\
\leq & \frac{\gamma\alpha_T}{q_{\min}} \sum_{j\in\mathcal{N}} \sqrt{T\sum_{t\in\mathcal{T}_j}||\boldsymbol{x}_t||^2_{\boldsymbol{V}_{j,t-1}^{-1}}} \\
\leq & \frac{\gamma\alpha_T}{q_{\min}} \sum_{j\in\mathcal{N}} \sqrt{2Td\log(\frac{\det(\boldsymbol{V}_{j,t-1})}{\det(\boldsymbol{V}_{j,0})})},
\end{aligned}$$

where the last inequality is due to Lemma 3. Obviously, $\det(\boldsymbol{V}_{j,0}) = \lambda^d$ and thus $\log(\det(\boldsymbol{V}_{j,0})) = d\log(\lambda)$. Denote the eigenvalues of $\boldsymbol{V}_{j,t-1}$ as λ_s. With the fact that $\det(\boldsymbol{V}_{j,t-1}) = \prod_s \lambda_s \leq (\frac{\text{trace}(\boldsymbol{V}_{j,0})+tL^2}{d})^d$ and $\text{trace}(\boldsymbol{V}_{j,0}) = \lambda d$, we have $\log(\det(\boldsymbol{V}_{j,t-1})) \leq d\log(\frac{\lambda d+tL^2}{d})$. Therefore,

$$\begin{aligned}
R_T \leq & \frac{\gamma\alpha_T}{q_{\min}} \sum_{j\in\mathcal{N}} \sqrt{2Td\log(\frac{\lambda d+TL^2}{\lambda d})} \\
\leq & \frac{\gamma\sigma N}{q_{\min}} \sqrt{2Td\log(1+\frac{TL^2}{\lambda d})} \\
& \cdot (\sqrt{2d\log(1+\frac{TL^2}{\lambda d})+2\log(\frac{2}{\delta})}+\sqrt{\lambda}) \\
= & O(\frac{\gamma\sigma dN}{q_{\min}}\sqrt{T}\log(TL^2/\lambda d)).
\end{aligned}$$

□